视频透视自适应校正系统

摘要

书本、纸张等平面目标在倾斜或旋转拍摄时会产生透视畸变，版面歪斜、文字拉伸，不能直接当作正视画面来阅读、归档或送入识别算法。而现有“拍照后逐张处理”的方式，在背景杂乱或目标移动时仍需人工框选四角，难以适应视频拍摄的场景。

本作品构建了一套基于 PYNQ 平台的视频透视自适应校正系统来解决这个问题。系统以目标的物理外边缘作为定位依据，把传统视觉和 AI 分成两个阶段配合使用：常规情况下先由传统算法检测轮廓、拟合四条边，并校验它们之间的几何关系。只有在常规搜索找不到有效目标时，比如目标边缘不清晰、颜色与环境接近，或边缘有轻微遮挡、反光，才调用轻量化分割 AI 模型给出候选区域和初始四角。候选四角通过当前帧验证之后，交由跟踪模块接管，此后不再逐帧运行神经网络。跟踪阶段用双向光流复测跟随目标运动，只在边缘证据不可靠或运动突增时才重新求解。

在软硬件分工上，控制、定位和跟踪放在处理系统 PS 侧运行，像素重映射、锐化和分段线性光补则由可编程逻辑 PL 侧承担。系统还通过相机复用、帧序号校验、异步任务和单一显示提交者这几种机制，减少了模式切换时的重复初始化，也避免了多路输出造成的画面不一致。界面上提供自动定位、AI 校正、手动四角、分辨率选择和调节参数（如锐化、光照补偿）等操作。当定位短暂失效时，系统会在限定时间之内继续用最后可信的四角去处理当前相机画面，而不是把画面冻结在旧图上。

系统已在 PYNQ-Z2 上完成部署并通过板端实测，累计运行数十小时、经历千余次交互会话未出现崩溃，能够持续稳定地完成实时校正与显示。

第一部分　作品概述

1.1 功能与特性

系统自动识别画面中近似平面的目标，定位其物理外边缘的四个角点，把倾斜、旋转或平移拍摄到的画面实时校正成端正、便于阅读的矩形，并且可以在"原图与校正图并排"和"只显示校正图"两种方式之间随时切换。默认情况下自动算法优先工作，遇到困难目标时由 AI 补充定位，用户也可以自己手动指定四个角点。输入支持 480P 和 600P 两个档位，另外还提供放大阅读、截图和相册功能。锐化和光补两个滑条拖动后立即反馈，参数统一在下一帧开始时生效。实时界面上同时显示当前校正结果的来源（AI 或传统算法）和输出帧率，方便判断这一帧是由哪一种方式定位的。目标运动时由光流跟踪逐帧跟随，不必逐帧重新搜索。

1.2 应用领域

透视校正现有的做法大致有三类：手机扫描类应用需要逐张手动框选四角；专用文档相机固定在支架上，靠机械结构保证正对；遥感和摄影测量里的几何校正依赖标定或已知控制点。三者都不处理“相机自由摆放、目标随时移动”的场景。本系统把定位、跟踪和像素级重映射串成一条实时链路，四角由算法自动给出，用户只在必要时介入。

本作品适用于桌面上的书本阅读、纸质资料展示、辅助阅读和嵌入式视觉教学；也可以用来跟随平板的外形或者呈现屏幕内容；还可以用在传送带上——相机斜装在一侧，对快速通过的快递面单这类平面目标持续跟踪四角，边移动边输出正面画面，供后续读码或识别使用。系统处理的对象是摄像机视野里的近似平面物体；对于平板这类设备，标注和定位覆盖的是包含黑色边框在内的机身物理外缘。边角轻微遮挡或轻微反光不影响定位，分割模型在训练时就覆盖了这类情况，仍然可以得到四角。至于书页弯曲、被完全挡住因而看不见的内容，以及强反光淹没的区域，这些不是单应性变换能够恢复的，系统不对此作保证。

1.3 主要技术特点

整条链路按"候选定位、当前帧验证、持续跟踪"三个阶段分工。传统轮廓与边缘方法负责快速定位，AI 分割在目标难以提取时给出空间先验。两者给出的四角先过几何约束，再由当前帧边缘证据确认，之后交给跟踪策略逐帧更新位置。PS 侧做事件调度，并解算几何参数；PL 侧承担大量重复的像素处理。神经网络不进入逐帧校正的主循环，CPU 占用因此降下来。异步结果都带帧序号、请求标识和页面代次，旧任务的结果不会串到新场景上。跟踪以双向光流复测为主，只在必要时才重新求解单应矩阵。

1.4 主要性能指标

表 1　交付版本的主要性能指标

指标	数值
四角定位耗时	自动检测 0.38 s（中位）；AI 候选返回 2.49 s（均值）
单帧跟踪耗时（全部帧）	自动 17.7 ms；AI 辅助 16.7 ms
光流复测	单次事务中位 71.6 ms
静态保持单帧	35.3～36.6 ms（480P 静止）
PL 单帧处理	自动 10.7 ms；AI 辅助 10.1 ms
平均帧率（单幅校正，静动态合计）	自动 23.2 fps；AI 辅助 21.1 fps
AI 角点误差	4.6 px
本地 AI 部署	MobileNetV3-Small + U-Net，约 137 万参数；输入 256×192 RGB；NCNN FP32
校正核吞吐	4.686 拍/像素（初版 11.99，提速 2.56×）
PL 资源	LUT 33.02%、FF 22.68%、BRAM 86.43%、DSP 20.91%
实现时序	WNS +0.650 ns、WHS +0.015 ns

1.5 主要创新点

（1）AI 与传统算法分工：传统轮廓负责快速定位，AI 只在困难场景补充区域先验；候选四角须经当前帧边缘证据验证后才交给光流复测，网络不进入逐帧主循环。（2）PS 与 PL 的分工与流水：PS 负责定位、跟踪与透视求解，PL 用全流式三级流水线做重采样与增强，两侧在同一时刻处理的是不同的帧——PS 解算下一帧的几何时，PL 正在重映射当前帧；同一比特流支持 480P 与 600P 两档。（3）光流复测按需触发：只有边缘证据不可靠、运动突增、漂移累积或锚点帧龄到期时才做一次双向 LK，触发后还要过四判据门控——四边齐全、最小支持率不低于 0.60、最大拟合残差不大于 1 像素、修正量不超过搜索半径的 0.65 倍；锚点到期做复验，而不是全图重搜。（4）物理外边缘统一口径：数据标注、模型输出与跟踪目标同取物理外边缘，平板含边框。（5）可证明的实时性：每帧提交前用定点区间算术证明行缓存访问不越界，不通过则自动分块或拒帧。（6）宽度换深度与整数域递推：512 宽省下的 BRAM 换 384 行深度，分块由 4 降到 1；坐标递推在精确整数域进行，每像素仅 3 次加法。

1.6 设计流程

开发按“三条线并行、两次交汇”的方式组织，并非一条线走到底。



图 1　三条研发线的并行推进与两次交汇

主线承担算法与硬件，分五段推进。定型解决“做什么”：在候选方案中做五维对比，确定平面自适应校正这条路线，形成定稿。打通解决“能不能跑”：从零建成 PL 校正核，覆盖块设计、流式重映射与坐标发生器，先让整条链路在板上运行起来。求对解决“算得对不对”：加入帧身份校验与色彩通路，把找四角从 PC 端搬到板上，以实拍样本判废率归零为通过标准。求快解决“连不连续”：把取帧、解算、重映射、显示四段做成重叠流水，并降低显示合成的开销。攻坚解决“够不够快”：对校正核逐轮压缩每像素拍数，同时缩减 600P 预览开销，最后完成交付打包。

第二条线承担交互与跟踪，在主线打通之后加入，分三段推进：先把触摸操作做成闭环，再让系统在静止与跟丢两种状态下都能自行恢复，最后把同帧成对显示和双档位切换做到不断流。这条线关心的核心是“在人手里用起来连不连续”。

第三条线是模型，在收口阶段加入，分两段：先补训练数据，再把轻量分割网络接进定位环节——只提供初始四角，拿到之后立即交回传统跟踪，不进入逐帧通路。

三条线之间有两次交汇。第一次，主线并入第二条线的代码库，此后“够不够快”与“连不连续”在同一份代码上同时推进；第二次，三条线合成同一个版本号，交付版本从这时起才正式存在。

全程的放行条件是同一条：先在测试平台上逐字节通过，再上板实测；任一条不满足，不进入下一段。

第二部分　系统组成及功能说明

2.1 整体介绍

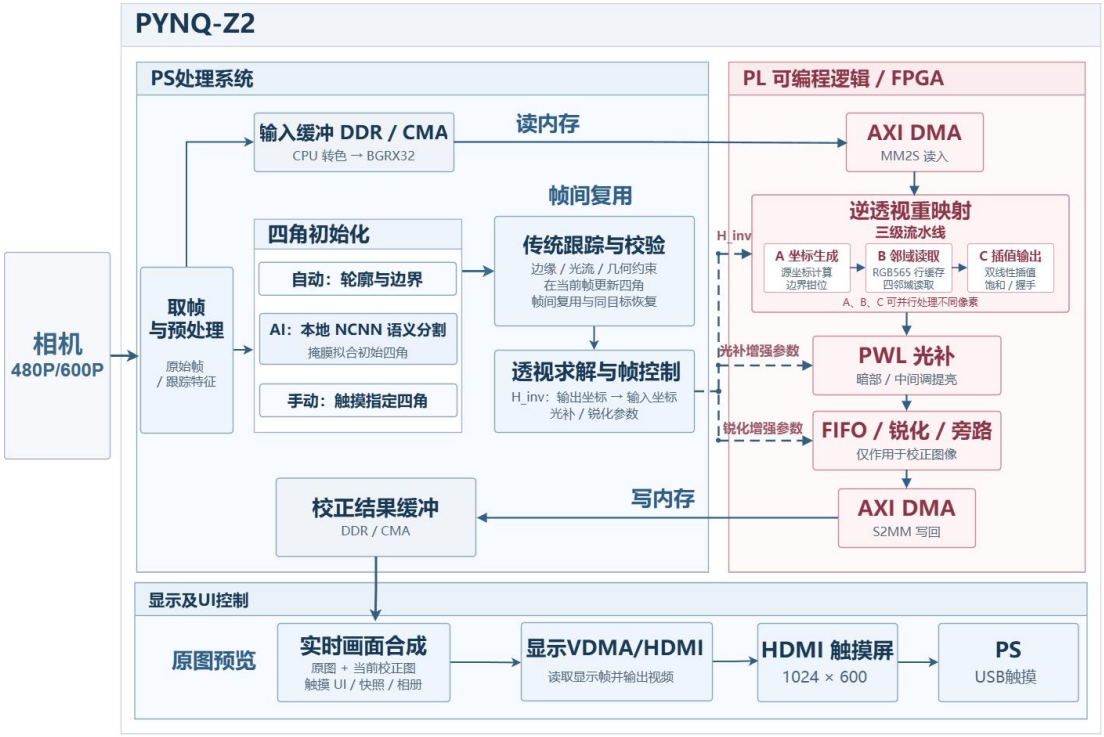


图 2　软硬件协作框图

相机按 480P 或 600P 采集，原始帧经取帧与预处理后由 CPU 转色成 BGRX32，写入 DDR 中的输入缓冲。四角初始化有三条路径：自动定位走轮廓与边界，AI 辅助调用本地 NCNN 分割网络、用掩膜拟合出初始四角，手动则由触摸指定。三者都汇入传统跟踪与校验，由四边证据、双向光流与几何约束在当前帧更新四角，并支持帧间复用与同目标恢复。跟踪结果交给透视求解与帧控制，算出把输出坐标映射回输入坐标的 H_inv，连同光补与锐化参数经 AXI4-Lite 下发到 PL。

PL 侧由 AXI DMA 从 DDR 读入像素流，依次经过三级流水的逆透视重映射、PWL 光补与锐化旁路，再由第二个 DMA 写回 DDR 中的校正结果缓冲。

显示环节把原图与当前校正图合成，经显示 VDMA 送到 1024×600 的触摸屏；触摸事件从 USB 回到 PS，只改变状态和参数，不另起第二个相机读取

者或显示写入者。

2.2 硬件系统介绍

系统以 PYNQ-Z2 开发板为核心，板载 XC7Z020（Zynq-7020，约 5.3 万个 LUT、140 块 BRAM），PS 为双核 ARM Cortex-A9，与 PL 通过片上 AXI 互联。外设是一只 USB 2.0 相机和一块 1024×600 的 HDMI 触摸屏，相机提供 480P 与 600P 两档，触摸经 USB 回连 PS。PL 主时钟 83.333 MHz，对外接口契约为 remap 1.5，输入格式 BGRX32；配套的 bit 与 HWH 文件描述了 PL 侧的外设、地址映射与接口。从 YUYV 到显示或处理所需格式的转换由 PS 侧的原生代码完成，PL 只负责像素重映射与画质增强。

2.2.1 PL 校正核与行缓存架构

校正核按照 A、B、C 三级流水线组织。A 级用"行首初值加行内递推"的方式生成源坐标并完成边界钳位，其中多路并行除法器保证了这一级的吞吐。B 级根据坐标 FIFO 给出的地址，在两个存储体上并行读出行缓存里的四个邻域像素；C 级完成双线性插值并把结果饱和输出。三级各自占用约 2 到 4 个时钟周期，稳态下整核的吞吐为 4.686 拍每像素。



图 3　校正核的 ABC 三级流水线结构（稳态吞吐 4.686 拍/像素，各级约 2～4 拍）

分成三级，是因为这几步之间没有真正的先后依赖。取坐标、边界钳位、读邻域、插值和输出如果串在一台状态机里，每个像素要花十来个时钟周期；其中行缓存读口只占四拍，其余时间都在等彼此独立的步骤完成。拆成并行的三级之后，稳态吞吐由最慢的一级决定，而不是所有级的耗时相加。

这里的"三级"指功能级的划分，每一级内部还有更细的状态层次。A 级包

含四个状态，坐标发生器内部是十六路并行除法单元加一个结果先进先出队列，输出比请求晚两拍到达。B 级用两拍读完四个邻域像素：偶、奇两个存储体并行工作，同一行左右两列在一个时钟内读出，两行因此只需两拍。C 级三个状态，其中等待状态占一拍，正好对上插值单元内部的两级寄存器延迟。插值单元必须拆成横向与纵向两级：合并到一拍时，综合报告给出的最差路径是 16.9 纳秒，在 10 纳秒周期下无法收敛。

A 级负责坐标生成，是整个核里最需要仔细处理的部分。如果每个像素都重新计算一次单应变换的六个乘法和三次加法，吞吐会被乘法器链拖住。因此采用"行首初值加行内递推"：每行行首算一次初值，之后向右每走一个像素，三个累加器各自只增加一个常量。这里有一个容易踩的陷阱：若把 Q16.16 的增量直接累加，每一步都会丢掉半个最低位，走完 1280 个像素后误差累积到 640 个最低位，相当于白白损失 0.01 像素精度。本设计改为在未移位的精确整数域里累加，只在最后输出时移位一次。

坐标除法的实现方式同样经过权衡。齐次坐标归一化本质上是一个逐像素除法。若顺序执行，一个坐标要花掉六十个时钟周期。本设计改为 N 路并行，各除法器按轮转顺序依次启动。由于延迟完全相同，完成顺序天然等于启动顺序，结果可以直接按序写入 FIFO，不需要额外的重排电路。这里的前提是除数必须在启动时锁存：并行度越高，输入总线换得越快，若除数直接取自活输入，同一组操作数算出的商就会偏。因此并行化之后，判据不能停留在"用了同一个子模块所以结果一样"的推理上，必须在不同并行度之间做逐字节比对。

B 级从行缓存中取出双线性插值所需的四个邻域像素。行缓存组织成环形的若干行，地址由"行号对缓存行数取模"再乘以有效行宽得到；为了在一个时钟内读出左右相邻两列，缓存按列地址最低位分成偶、奇两个体，各有一个独立读口。总容量仍然是行数乘以有效行宽，没有多占一块 BRAM，只是把读带宽翻了一倍。当缓存行数不是二的整数次幂时，不能直接写取模表达式让综合器生成通用取模电路，那样会在地址路径上引入组合逻辑，本设计改用预计算加条件减法完成回绕。

C 级完成双线性插值。三个颜色通道走并行的数据通路，而不是分时复用同一套乘法器：复用虽然能省 DSP，却会把每像素延迟从两拍拉长到六拍，对吞吐的伤害远大于资源上的收益。插值结果经饱和处理后输出，并在最后一个像素上给出帧结束标志，与行缓存的读写地址共同构成流控握手。

参数通过一组 AXI-Lite 寄存器下发，地址空间 0x00 到 0x30。0x00 控制，0x04 状态，0x08 与 0x0C 写入输入和输出尺寸，0x10 到 0x2C 共八个寄存器存放 Q16.16 的单应系数，0x30 是只读版本寄存器。软件每帧下发参数后会把十个寄存器全部读回比对，不一致立即报错。版本寄存器让软件在启动时确认，板上加载的比特流是否与预期一致。

校正核看起来只有一条固定的数据通路，是因为大部分设计空间在开发过程中已经被排除掉了。表 2 列出了当时认真考虑过、最终没有采用的四个方案及其实测代价。其中最直观的是整帧缓冲一项：完成同样的重映射，核内行缓存方案耗时 21.9ms，而需要读写整帧的参考实现为 68.9ms，多出来的时间几乎全部花在搬运上。核的"简单"是逐项量化之后收敛出来的结果，而不是出发点。

表 2　被排除的设计方案与实测代价

若采用	代价	最终方案与实测
整帧缓冲代替行缓存	640×480 的 YUYV 帧约 600KB，超过整片 BRAM 的余量，只能落在 DDR	改为 O(行) 行缓存：核内 21.9ms，落 DDR 的参考实现 68.9ms
每像素重算单应坐标	每像素 6 次乘法 +3 次加法，约 60 个时钟周期	行首初值 + 行内递推，坐标生成降到 3 次加法
坐标除法顺序执行	每个坐标约 49 个时钟周期	16 路并行、轮转启动，降到 3.06 拍/坐标
行缓存分 4 体加宽读口	BRAM 96 → 128 块（91%），布局布线不通过；吞吐仅 7.160 → 7.162	按列低位分 2 体：容量不变，四邻域 2 拍读完

除了性能，校正核还承担一项容易被忽略的责任：行缓存的访问必须是安全的。环形行缓存允许写指针回头覆盖最老的一行，前提是读侧永远不读比缓存深度更老的数据。一旦目标倾斜角变大，一行内需要的源行跨度可能超过缓

存深度，读侧就会拿到被覆盖的内容，画面表现为无法解释的色块。本设计不靠"参数开小一点应该不会超"这类经验判断。每帧提交之前，用定点区间算术沿所有输出像素把源坐标的取值范围推一遍，证明最深的一次访问仍在缓存深度之内。不通过时自动把该帧拆成更小的分块，拆分后仍不通过则拒绝这一帧。

从系统约束看，PL 侧也不需要更复杂的结构。USB 2.0 实际可用带宽约 35MB/s，640×480 的 YUYV 帧在 30fps 下已占去 18.4MB/s。继续提高分辨率或帧率，都会被这条链路卡住。校正核的处理能力是 17.8 兆像素每秒，相机只能提供 9.2 兆像素每秒，余量接近两倍。当前设计的 BRAM 也已用掉 86.4%，没有余量再开第二条并行通路。所以更合理的方向是把这一条通路做到底。

行缓存按实际输入宽度 512 像素、深度 384 行分配，格式为 RGB565。裁剪后的实际宽度不超过 328 像素，若仍按 1024 的上限分配，约 37.5% 的 BRAM 会长期闲置。把这部分宽度让给深度之后，整帧的源行跨度可以一次装下，分块数由 4 降到 1；每个分块的证明、参数下发和 DMA 启动开销也随之消除。

2.2.2 图像增强与输出链路

校正核之后是图像增强。两级都走 AXI-Stream，按像素流处理，不做整帧缓存。光补采用分段线性的暗部提升，因此不需要为局部直方图分配大块缓存，硬件上占 0 BRAM、1 DSP。锐化由现有的 PL 模块实现，效果受单块输出宽度与拼接边界策略约束。目前的亮度映射是全局的，还没有做到空间局部自适应。

2.3 软件系统介绍

2.3.1 软件组织与数据流

板端软件为单进程多线程结构，按职责分为五层：会话与采集、传统检测、AI 服务、混合调度与跟踪、硬件与显示，各层的输入输出见表 2。采集线程负责取帧并打上帧序号与页面代次；检测与 AI 给出候选四角，调度层完成

验证与跟踪；硬件后端把透视参数下发给 PL，收回校正后的图像；显示线程是唯一的 HDMI 提交者，按当前模式把原图与校正图合成输出。AI 服务常驻独立线程，只在需要时被唤醒，推理结果带请求回执，过期结果直接丢弃。

表 3　软件模块及其主要输入输出

模块	主要输入	主要输出
采集与会话	相机档位、页面事件	图像、帧序号、页面代次
传统检测	当前图像、点选位置	候选四角、边缘及几何证据
AI 服务	请求图像、目标选择上下文	分割掩膜、候选四角、请求回执
混合调度与跟踪	候选、当前帧、历史可信四角	跟踪四角、来源、有效状态
硬件与显示	映射参数、当前像素、增强参数	校正图、原图、FPS 与进度

2.3.2 传统自动定位

自动路径利用亮度、色彩和边缘信息产生轮廓候选，再把这些近似四边形进一步精化为目标的物理外缘。轮廓拟合本身只是提出候选，之后还要检查凸性、面积、边长关系以及当前帧上的四条边证据，防止把背景纹理或者书页里印刷的矩形当成外框。与单纯挑选最大掩膜的做法相比，轮廓候选能够提供多个目标之间的边界分离信息。[1]

自动搜索的设计目标是尽量优先拿到低延迟的定位机会。在当前的恢复控制逻辑里，完全没有候选时的搜索时间是 1.5 秒，已有候选的情况下总验证时间上限为 2.8 秒；只有超过了这个阶段预算，才会进入带有请求次数限制的 AI 补充定位。上面这些数值只是控制条件，并不代表每次切换实际需要的总时长，真实耗时还包括任务调度、图像准备、推理和当前帧验证等环节。

2.3.3 AI 与传统算法的融合机制

AI 接收的是经过统一预处理的 256×192 RGB 图像，分割模型输出目标所在的区域。后处理环节把掩膜转换成几何候选，再结合用户的点选位置、物理边界以及当前帧验证来确定最终目标。AI 给出的候选并不直接等同于可以输出的校正结果。当多本书的掩膜连在一起、边界本身就很模糊，或者目标在跨帧之间发生移动时，仍然可能产生把几本书一起框住的大框，或者拿到过时的四

角。所用模型为 MobileNetV3-Small 编码器加轻量 U-Net 解码器，共 143 层，权重 5.46MB[2-3]。推理采用 NCNN FP32，在 ARMv7 上单线程运行，实测单次耗时中位数约 28.5ms[4]。模型进程常驻内存，只在需要时被调用，不参与逐帧处理，因此不会拖慢正常的校正输出。



图 4　轻量分割网络的完整结构与一次真实推理样例

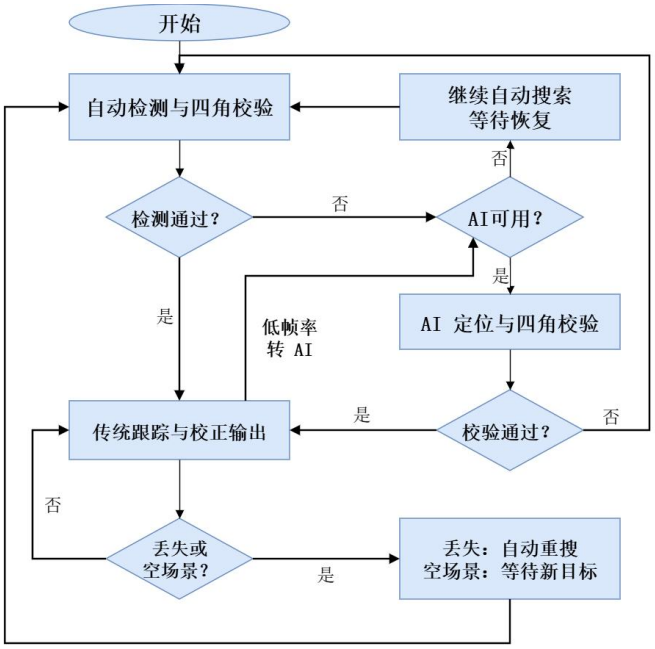


图 5　自动检测、AI 补充定位与传统跟踪的调度关系

传统方法与 AI 的融合体现在三个层面上。第一个层面是搜索调度，常规场景先交给传统方法处理，遇到困难场景才按需启用 AI。第二个层面是候选验证，AI 提供的是区域先验，具体边界还要由传统边缘和几何关系来约束。第三个层面是时间上的连续性，AI 一旦找到目标就不再逐帧推理，而是把初始四角交给后续的跟踪模块，之后是否需要恢复由跟踪的质量来决定。

用户主动点击 AI 校正后，在还没有找到目标之前，系统会一直保持这次 AI 定位的意图；目标被找到并正常输出之后，即使出现短暂失效也先延续当前画面，直到满足原有的丢失条件才退回自动恢复流程。

多本书场景下的点选，需要在 480P 和 600P 两个档位里都统一把显示坐标换算成相机坐标，然后把选中的目标意图传给 AI。需要注意的是，包围框包含点击点只能说明这一下点在了框内，并不能证明选中的就是某一本书；如果无法可靠地拆分，系统应当报告"选择未确认"，而不能把几个目标的大包围框当成选择成功。

2.3.4 跟踪与恢复控制

跟踪模块的输入是当前相机图像、已经确认的四角以及同一目标的参考信息。系统沿着目标的物理边缘建立证据，再结合特征跟踪、前后向一致性和几何验证，判断目标运动之后给出的四角是否仍然可信。对于白书放在白底上或者纹理很弱的平板这类情况，只靠纹理点并不足以获得可靠的约束，因此还要同时利用四条边的观测结果和目标内部的参考信息，防止跟踪结果吸附到背景纹理或者平板内屏上。

光流跟踪不逐帧运行，只在四种情况下触发：边缘证据不可靠、目标运动突然加剧、累积漂移超限，或锚点帧龄到期。触发时用当前帧与锚点帧做一次双向 LK，求出的位移要通过下面的门控，才允许重新解算单应矩阵。

LK 使用 21×21 的窗口。快速路径的金字塔最多 3 层、迭代 12 次；全路径最多 4 层、迭代 20 次，位移收敛阈值 0.02 像素。目标外缘上最多取 100 个特征点，每个点的前后向误差小于 1.4 像素才计入；同一个锚点上，前后向计算始终使用同一图像通道。

两条路径按运动幅度选择：小运动先走预测快速路径；锚点位移超过 12

像素、或采集步长不小于 6 帧时直接走全路径；只有在快速路径求解失败、且时间预算仍允许时，才升级为全路径。

门控由 edge_gate 判定，要求四条边齐全、最小支持率不低于 0.60、最大拟合残差不大于 1 像素、修正量不超过搜索半径的 0.65 倍，并且四条边的极性一致。锚点本身有帧龄上限，最大间隔 30 帧；到期做一次复验，而不是直接全图重搜。静态保持期间还要受可信期限的约束。

有四种情况会拒绝更新：有效点少于 4 个、RANSAC 内点率低于 0.65、角点越出图像边界，或者当前帧的物理边缘复核不通过。被拒绝时不更新位置，直接交给恢复逻辑；局部恢复失败之后才扩大搜索范围，避免对同一个候选反复验证。

出现短暂失效时，系统最多延续 1 秒，用最后一次可信的四角去校正当前相机画面，左右两路仍然保持更新。这个计时从最后一次可靠确认开始算起，中间靠估计得到的帧不会反复延长可信期限。一旦确认目标已经离开、换成了另一本书，或者运动幅度大到无法可靠跟随，旧的位置就会被清除。确认丢失之后，系统采用 1.2 秒的确认条件和自动优先的恢复顺序。自动 AI 的失败请求仍然受预算约束，而在成功并稳定 3 秒之后，可以根据现有证据退还这次请求的名额。

2.3.5 平面透视校正

系统把有顺序的四个源角点与输出矩形的四个角建立对应关系，据此求出这个平面的投影映射[5]。PL 侧采用逆向映射，为每一个输出像素反查它在输入图中的位置，这样可以避免正向映射时散射造成的空洞。选择"原比例"输出时，画布布局由目标本身的几何关系决定。由于四角一旦出错就会直接改变映射并造成画面拉伸，定位与跟踪结果的可信程度，比在显示端再加一层滤镜更为关键。

2.3.6 增强参数、帧率与并发管理

增强参数按最新值优先，最多每 100 毫秒更新一次，并在下一帧开始前统一生效；调整时不需要重启相机，也不需要重新找角或改变定位模式。

帧率只统计新的有效校正帧成功提交到 HDMI 之后的情况，重复重绘、冻结画面和仅显示原图都不计入。交付版本取最近 3 秒的计数除以 3，每 100 毫秒刷新一次。这个指标反映的是有效输出吞吐，既不等于面板的物理刷新率，也不等于单帧耗时的倒数。

并发方面，AI 服务由独立线程负责加载、推理与退出，结果带代次与帧回执，过期结果直接丢弃；相机复用省去反复打开设备的开销，但改变分辨率仍要重配视频流并重新定位；系统始终只保留一个 HDMI 提交者，队列长度有限，避免多路显示争用与任务积压。

第三部分　完成情况及性能参数

3.1 整体介绍与工程成果

本系统基于 PYNQ-Z2 平台实现，PS 端完成相机采集、目标检测、四角跟踪与单应矩阵规划，PL 端实现光补、锐化与重映射校正的硬件流水线，校正画面经 HDMI 输出。系统实现了传统自动校正与 AI 校正的协同，同时包含手动校正功能。自动校正通过边缘检测与光流跟踪实时估计目标四角，本地神经网络会在传统校正过慢或失败时接管校正。校正过程中实现了运动中实时校正、放大阅读、画面冻结与截图等功能；图像处理中实现了输入画质切换（480P/600P）、光补、锐化。为改善人机交互体验，系统采用全触摸屏操作界面，全部操作均可通过多点触控完成。

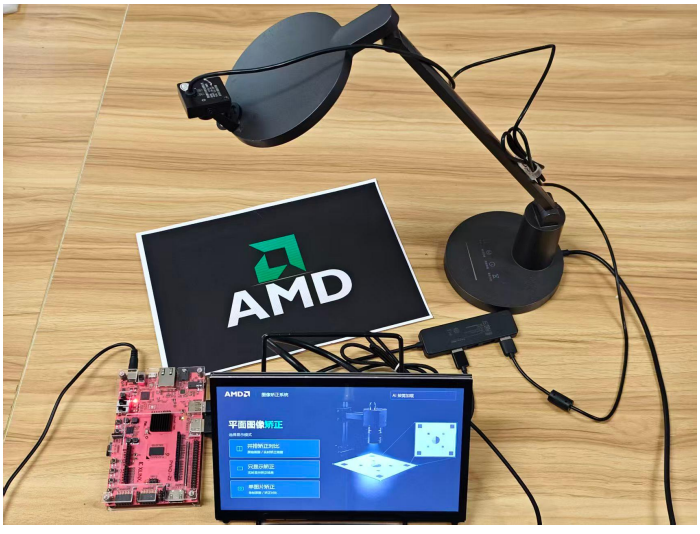


图 6　完整作品：PYNQ-Z2 板卡、7 寸触摸一体屏、台灯臂夹持的 USB 摄像头

软件层面，完成 MobileNetV3-Small + U-Net 轻量分割模型训练与 NCNN FP32 板端部署，模型约 137 万参数。构建传统检测优先、AI 辅助校正、统一跟踪更新的协同机制：传统检测超时或失败时调用 AI，分割结果经几何拟合与当前帧边界校验后交由跟踪器持续更新，避免逐帧运行网络推理。同时实现目标点选、丢失恢复、模式调度、运行日志与性能统计。

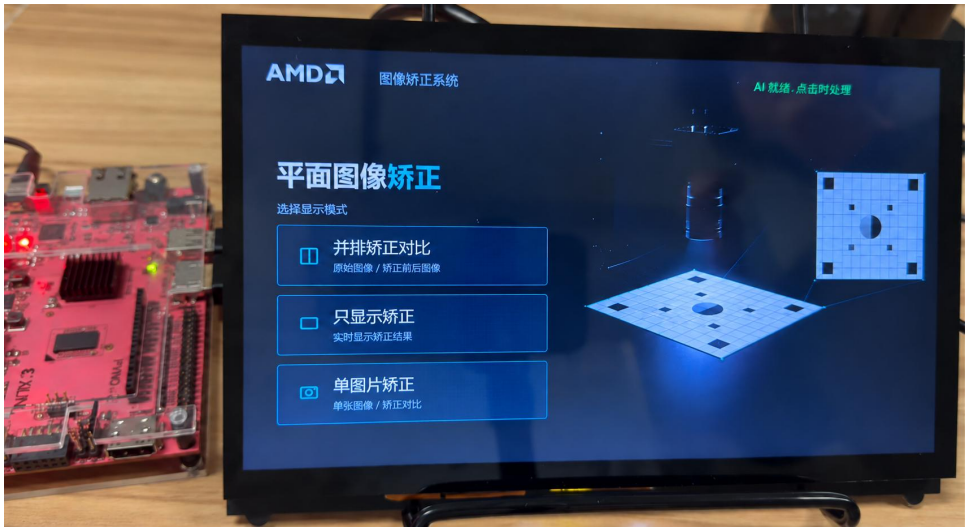


图 7　系统首页

3.2 特性成果

3.2.1 传统自动校正

自动校正的效果如下图，并排校正帧率可达 19 帧/秒。



图 8　并排自动校正对比实拍

3.2.2 手动校正

当用户对系统自适应校正的结果不满意时，可随时进入手动校正模式。用户在触摸屏上依次点选目标的四个物理外角，支持确认、撤销、重标与放大辅助选点，确认后立即计算单应矩阵并经 PL 完成校正输出，与自动校正共用同一校正流水线。

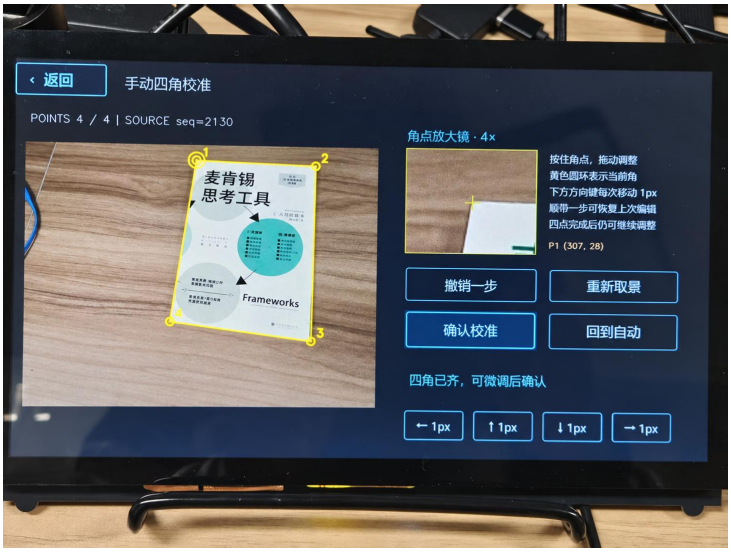


图 9　手动校正界面

3.2.3 放大阅读

系统提供放大阅读功能，用于细读校正画面中的小字。放大窗口支持两种拖动方式：在左侧总览图上直接拖动，焦点随手指移动；在放大窗口内拖动，则按当前倍率平移观察范围，实现拖动式阅读。放大支持 2 倍和 4 倍。

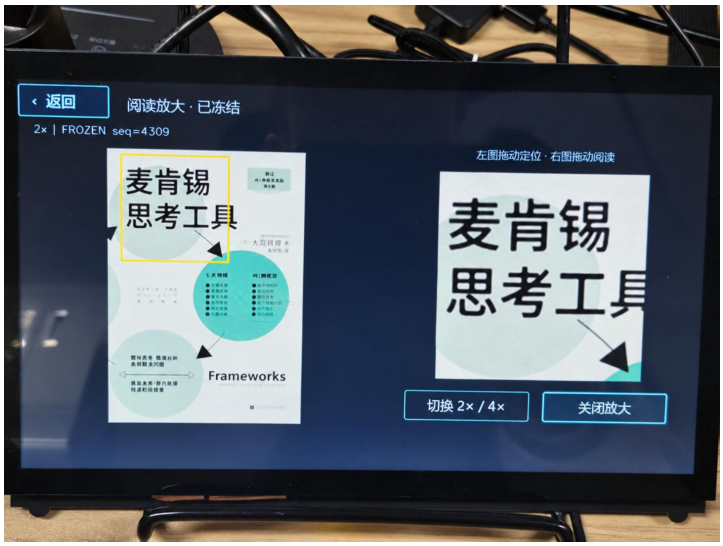


图 10　放大阅读演示

3.2.4 显示比例切换

系统提供显示比例切换功能，用于按书页、平板等阅读对象的实际形状调整校正输出画面的比例。点击界面右上角「比例」按钮打开右侧设置面板，面板提供「原始、1:1、3:4、4:3、16:9」五种预设及「自定义」比例。



图 11　显示比例切换

3.2.5 多目标切换

当画面中存在书页、平板等多个平面目标时，系统支持通过点击直接切换校正目标。若点击位置仍落在当前目标四角内，则判断为重复点击并保持当前跟踪，不触发切换。

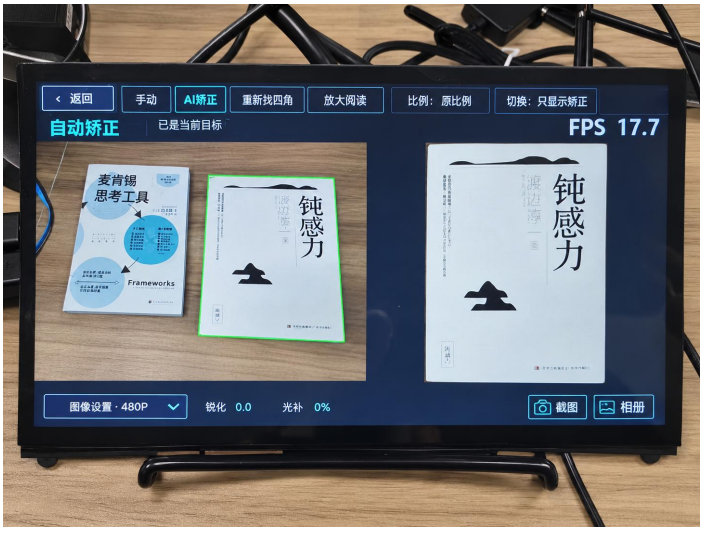


图 12　多目标切换

3.2.6 校正图像处理

系统在 PL 校正流水线中集成了光补与锐化两种图像增强功能，用于改善拍摄光照不足或画面偏软时的阅读效果。光补采用 PL 端分段线性（PWL）暗部提升算法，强度可在 0～100% 之间调节，提供 0、20、40、60、80、100 六个预设档位及连续滑条。锐化采用 PL 端流式锐化内核，强度可在 0～4.0 之间调节（共 40 级），提供 0、1.0、1.5、2.0、3.0、4.0 六个预设及连续滑条，用于增强文字边缘的清晰度。



图 13　图像处理

3.2.7 相机分辨率调节

系统提供 480P 与 600P 两档相机输入分辨率切换：480P 为 640×480@30fps，600P 为 800×600@15fps。

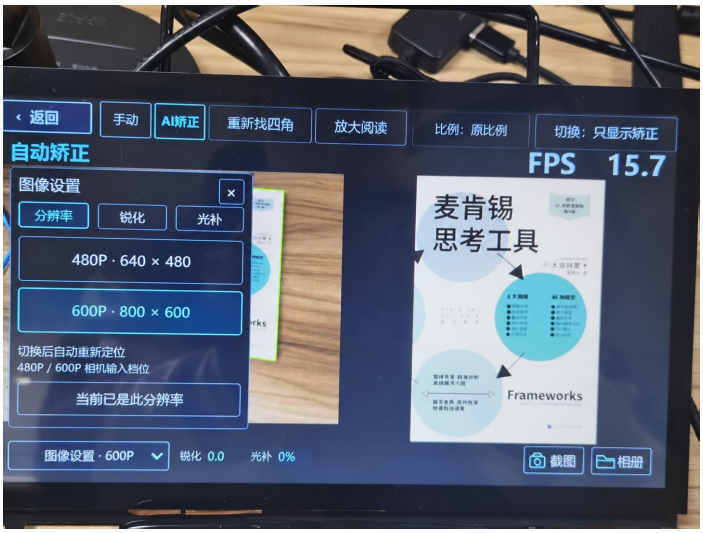


图 14　相机分辨率调节

3.3 训练数据与模型验证

模型沿用此前训练完成的结果。数据包括历史样本 406 张、PIC7 共 27 张、PIC8 共 36 张，合计 469 张，其中训练集 379 张、验证集 36 张、测试集 54 张。训练共 80 轮，选定最佳轮次后冻结参数，再在测试集上评估。PIC7 和 PIC8 都进入了训练组，所以它们评估时的改善只能说明模型拟合了这些样本，不能当作独立泛化的结论。

表 4　模型在历史验证集与各数据组上的表现

数据范围	旧模型通过	新模型通过	平均角点误差
历史验证集	36/36	36/36	3.90 → 3.67 px
保留测试集	53/54	53/54	4.63 → 4.29 px
PIC7 训练样本	14/27	27/27	123.31 → 1.91 px
PIC8 训练样本	34/36	36/36	6.55 → 1.66 px

定位的通过条件是：在 640×480 坐标系下，四角平均误差不超过 10px，单角最大误差不超过 20px。评估器会把未检出的样本按 999px 计入均值，所以报告里看到的误差大幅下降，有一部分来自漏检变少，不全是定位变准。[7]

3.4 实时运行与验证边界

板端的运行记录来自同一份 run-result.log，共 56,606 条。这份日志只覆盖开启记录功能的时段；拍摄演示与部分现场测试为了降低开销没有开日志，所以下面的数字是下限，实际运行时长和交互次数都高于此。其中有记录的运行累计 4.8 小时、18 次程序启动、232 次交互会话，共处理 79,073 帧，其中 65,673 帧为有效校正输出；其余帧大多来自画面里没有目标、或正在切换目标的过程。崩溃捕获机制在每次启动时均已装载，运行期间没有捕获到任何信号。有记录的最长一段无中断连续运行约 49 分钟。

同一份日志里还能看到早期版本的一次板端运行：画面先用 600P 再切到 480P，并切换过双画面与单校正两种显示方式。[8]

表 5　早期版本与交付版本的输出帧率对照

场景	输出 FPS 采样范围	说明
早期版本 600P 双画面	11～19	包含两次双画面显示阶段
早期版本 480P 双画面	16～21	剔除刚获得输出的窗口建立点
早期版本 480P 仅校正	16～27	21 个采样点，中位数 25
交付版本实测 480P 并排	6.5～19（中位 8.0）	285 个采样；p90 14.5，最大 18.0
交付版本实测 480P 仅校正	18～27（中位 17.8）	156 个采样；p90 25.5，最大 26.5

表格中前四行来自早期版本日志，只作历史对照；后两行是对交付版本按相机序号逐帧重新统计的结果，口径为“有效输出帧数除以实际耗时”。

并排模式与单校正模式的帧率差距来自显示合成，不在定位或重映射。并排模式每帧要同时合成原图与校正图两路并叠加界面，实测合成耗时中位数 39.4ms、均值 45.2ms。单校正模式只合成一路，中位数 20.6ms、均值 23.5ms，两者相差约 20ms，这正是两种模式帧率差的主要来源。此外，计算线程与显示线程同处一个 Python 解释器，共享同一把全局解释器锁。当计算侧持续占满时，显示合成被迫让出执行机会，实测出现过单帧处理时间变短而并排帧率反而下降的情况。

静态段与动态段的帧率差别，来自跟踪是否触发复测。目标静止时系统复用上一步的几何结果，单帧处理中位 35.3～36.6 毫秒；目标运动时要逐帧做光流复测，复测帧的整帧中位升到 142.5 毫秒，其中 LK 本身占 71.6 毫秒。所以静态段帧率明显高于动态段，动态段又会随目标运动幅度波动，表 1 里的平均帧率是这两段合起来的数。

早期还做过一版取舍：把动态帧率拉平，不让它随运动下降。代价是动态段的画面跟随明显变差，目标一旦移动，四角更新滞后，校正画面出现拖影。两版对比之后选了现在这一版——动态段优先保证跟得住，静态段利用保持把帧率拉高，而不是把两段的帧率做成一样的数字。

光流跟踪在这套门控下是稳定的。复测只在必要时触发，触发之后只有两种结果：通过门控就更新位置，不通过就判失败交给恢复，不会停在中间状态；没有触发复测的帧由四边证据直接确认，不额外消耗算力。

3.5 PL 校正核的设计迭代与验证

校正核的吞吐在整个开发过程中经历过四轮迭代，从流水线化前的 11.99 拍每像素降到最终的 4.686 拍每像素，累计提升 2.56 倍。每一轮改动之前，都先用片上性能计数器把各流水级的占用逐拍拍开，确认瓶颈在哪一级，再动手改。

表 6　校正核吞吐的四轮迭代

轮次	主要改动	拍/像素	累计相对流水线化前
流水线化前	串行状态机，取坐标、读邻域、插值逐拍完成	11.99	1.00×
第一轮	坐标流水级独立成级，三级真正并行	7.160	1.67×
第二轮	坐标发生器除法并行度 8 → 16	6.675	1.80×
第三轮	行缓存按列地址低位分两体，读邻域 5 → 3 拍	5.679	2.11×
第四轮	插值级状态 5 → 3，去掉本级的邻域锁存	4.686	2.56×

第一轮把负责坐标的流水级也拆成独立的级，使三级真正并行，每像素从 11.99 拍降到 7.160 拍。第二轮把坐标发生器的除法并行度从 8 提到 16，除法环节的占用从 6.125 拍降到 3.06 拍，整核随之降到 6.675 拍。第三轮把行缓存按列地址低位分成两个体，使左右两列的读取可以并行，同时压缩了读邻域那一级的状态数，整核降到 5.679 拍。第四轮把插值级的状态机从五个状态压缩到三个，去掉了这一级自己的邻域锁存，改成当拍直接送给插值单元，最终降到 4.686 拍。

除了吞吐，还有一轮是为时序收敛做的。插值单元最初把横向与纵向插值

合在一拍里完成，综合报告给出的最差路径是 16.9 纳秒，在 10 纳秒的周期下严重违例，WNS 为 −6.878 纳秒。拆成两级流水之后收敛到 +0.488 纳秒。这一轮不改善吞吐，反而让每像素多花一拍，但没有它就上不了 83.333 MHz 的主时钟。

值得一提的是，第一轮之后的瓶颈判断曾经出过错。当时看到“读邻域那一级等待上流的时间占了 5.2 拍”，便认定它是瓶颈，于是把行缓存分成四个体、把这一级从五拍压到两拍。十七个测试平台全部逐字节通过，功能完全正确；但吞吐只从 7.160 变成 7.162，几乎没有变化，而 BRAM 却从 96 块涨到了 128 块，占整机的 91%，最终布不通。回头再看才发现，瓶颈并没有消失，只是从“等前一级”搬到了“等后一级”：这一级变快之后，后面插值级的等待时间立刻涨了回去。这个教训后来固化成一条规则：改动之前先列出所有并行级各自的每像素占用，取其中的最大值，并预先算出改完之后这个最大值会变成多少；如果最大值不变，这次改动就不会带来任何吞吐收益。

同样被证伪的还有两个参数取值。一是把除法并行度取成 12，算术上够用也省资源，但非二的整数次幂会让仿真永远跑不完。二是把行缓存深度加到 448 行，可以让两分块变成一分块，综合探针却显示需要 128 块 BRAM，占整机 94.3%，同样布不通。这两个参数最终都退回到二的整数次幂与已验证的 384 行。

功能正确性的验证采用了双参考判据。每一级 RTL 都有一份独立的定点行为模型，测试平台同时比对两份参考，任何一方的差异超过零都会被判定失败。目前共十七个测试平台参与回归，覆盖单块与多块、横向与纵向、边界钳位与饱和输出等场景，逐字节一致的结论是每次改动的放行条件。这套判据在四轮迭代里一共抓出三个只有在特定几何姿态下才会触发的缺陷，其中两个是“综合能过、仿真不过”的类型，靠上板试是试不出来的。

表 7　优化前后的性能与资源对比

项目	起点（彩色首版）	校正核改完	交付版
校正核吞吐	11.99 拍/像素	7.160 拍/像素	4.686 拍/像素
Block RAM	96 / 140（68.6%）	100 / 140（71.4%）	121 / 140（86.43%）
LUT	8,539	15,959（30.0%）	17,569（33.02%）
DSP	15 → 29	40（18.2%）	46（20.91%）
WNS	−6.878 ns	+0.488 ns	+0.650 ns

第四部分　大模型协作与技能包

4.1 智能体工作流

项目采用团队主导设计与验证、智能体辅助分析与实现的协作方式。团队提出应用需求、性能目标和资源约束，结合实机结果决定方案取舍；大模型在明确划定的范围内辅助编写代码、分析报告和执行构建。工作流分三层。

第一层由团队明确需求与技术边界。帧率优先、物理外边缘、H 估计降级链和接口约定这些要求，经多轮讨论和实机反馈逐步固定，再借助大模型整理进项目指令文件，让后续实现始终遵照已确认的设计。

第二层把设计要求转化为可复核的验收条件。仿真与定点模型检查数值，综合实现检查关键告警与 WNS，板端对照检查实际输出与帧率。团队结合真实失败场景确定验证重点，大模型辅助生成测试、执行检查并整理差异；代码能生成不等于任务完成。

第三层按确认后的任务调用工具链。团队拆分问题、限定修改范围，大模型辅助运行 Tcl、仿真、综合实现与数据分析，完成代码和文档更新；团队再用板卡、相机与触摸屏检查实际效果，依据结果决定保留、回退还是继续改。

团队的工作贯穿全过程，包括系统方案与关键取舍、硬件连接与调试、困难样本采集、四角标注复核，以及不同背景、光照和运动条件下的反复测试。现场发现的跳框、闪屏和切换等待等问题，会持续反馈到下一轮迭代。

4.2 提示词与自我纠错

提示词是随着测试和复盘逐步细化的。早期只笼统要求「优化这个模块」，很难得到可验证的方案；后来把任务写成「列出各并行级每像素占用，预估改动后的瓶颈，再判断是否值得实施」，并要求一并给出资源代价与对照数据，讨论才能落到具体取舍上。

纠错靠实测反馈与代码分析相互印证。举例来说，PL 输出整体错位一格时，先验握手时序、再验行缓存深度，两次改动都没有变化；随后核查实际参与综合的源码，才发现改的是没有被引用的副本。实机现象、构建记录和代码必须对得上，模型给出的假设也要逐条验证。

这些复盘沉淀成「先验证前提、再继续推理」的协作规则，并写进了项目指令。开发记录保留了九个纠错案例，包括性能估算偏差靠分段计时纠正、陈旧状态位误导排查方向、以及连续两个假设被证伪，供后续方案审查时对照。

4.3 技能包的提炼过程与边界

团队把项目中有效的方法归纳为提示词工作流、案例模板、校验脚本和踩坑清单四类，大模型协助整理结构、补齐说明、统一格式。

条目都从实际调试里提炼：团队给出故障现象与运行条件，配合模型完成复现、定位和修复，再补回归用例。流式握手的条目对应输出错位、下半图拖影和 DMA 挂起三次上板故障；行缓存深度的条目来自估算与实测相差三十倍的那次复盘，两者都保留了具体证据。

有效性用两种方式检查：撤掉修复看问题能否重现，以及从零跑一遍完整构建；最后再结合板端效果复核。技能包沉淀的是团队与智能体共同验证过的方法，以及它们的适用边界。

边界也写明：不含完整算法实现，不含 HLS，不含高速接口，不含多板卡经验；全部经验来自 Xilinx Zynq-7000 与 Vivado 2025.2，换大版本后部分报错信息会变。

4.4 AMD ROSS 智能体套件的使用心得与优化建议

AMD ROSS 套件里的官方 Vivado MCP 从 10 月 2 日起接入本项目的工具链，走 TCL 模式，不启动图形界面。它主要承担需要真正动 Vivado 的工作：启动会话、跑 Tcl、复核交付物、跨版本重建工程。两条可复现的量化记录：启动 Vivado 并跑通第一条 Tcl 用时 14.2 秒，同样的事手工开界面约需 3 到 5 分钟；打开已交付工程读取实现报告用时 77 秒。

用下来体会最深的是三点。一是会话常驻，Vivado 进程不随对话结束而退出，后续命令不必反复冷启动，这一条在连续调试时省得最多。二是长命令会自动交接，返回 running 状态后可以继续做别的事，不像图形界面那样整块占住。三是没有界面也能拿到全部现场，关键告警原文、端口清单与 IP 锁定状态都直接返回，排查时不用截图对照。

需要说明的是，MCP 省的是敲命令和等界面的时间，不是想清楚怎么做的时间。设计决策、参数取舍与判据设定仍然由人完成。

给套件的几条建议都来自实际踩到的点。第一，跨版本打开工程时（2026.1 的工程在 2025.2 上打开）返回的是只读，需要自己想到用 save_project_as 降级另存，建议在打开阶段就把这个提示打出来。第二，工程引用的 IP repo 缺失时，报错要到综合启动才暴露，而且表现为顶层被自动替换成别的模块，排查成本很高；建议在打开工程时就校验 IP repo 是否可达。第三，长命令目前只有 running 和 success 两个状态、没有进度，而综合实现动辄几十分钟，若能透出 Vivado 自身的进度百分比会更好用。

还有一处用法值得单说。判断行缓存要多深才够时，没有去跑整个设计的综合，而是用 create_project -in_memory 加 synth_design -mode out_of_context，在一个会话里连跑九个配置，把「BRAM 够不够」从推算变成了实测。这类只针对单个模块的问题，走 OOC 比跑全设计快一个数量级，而且结论不受其他模块牵制。

第五部分　复现说明

复现分两条路径，按要做的事选一条。环境统一为 PYNQ-Z2

（xc7z020clg400-1）、板上 PYNQ v3.1 镜像（pynq 3.1.1 / Python 3.10.4），EDA 用 Vivado 与 Vitis 2025.2；交付的位流就是在这套环境下由仓库脚本建出来的，按路径 B 可重建出同一设计。

路径 A 直接上板，约 15 分钟。上板对象是整个发布目录，共 115 个文件、26.5 MB，不能只挑其中几个——启动时会按清单逐个核对顶层 94 个文件的哈希，缺一个或内容不一致就拒绝启动。上传脚本先把文件写到临时目录，逐文件核对后再就地覆盖，不一致则中止并保留旧目录。硬件要求四项：PYNQ-Z2 板卡、接在 HDMI OUT 的显示器（输出固定 1280×720）、固定为 /dev/video0 的 UVC 摄像头，以及支持绝对坐标的 USB 触摸屏；缺触摸屏时启动会直接报错。板上打开 Jupyter，进入该目录运行启动 Notebook，先重启内核再依次执行前两个单元；运行时点一下触摸屏开始处理，界面显示帧率读数，默认运行时长上限两小时。

不开相机也能先验一套：运行 AI 自检 Notebook，用包内 10 条用例图跑一遍分割，全部通过即说明 AI 子进程、清单校验与图片素材都完整。

路径 B 从零重建，约 1 小时，另加综合与实现的时间。PL 侧用 Tcl 脚本从 Block Design 源、7 个已打包的 IP 与 RTL 建工程，再依次跑综合、实现、写位流，导出 9 份报告与硬件描述文件；放行条件是 WNS 大于 0、DRC 错误为 0、布线错误为 0。PS 侧源码为纯 Python，无需编译；6 个 C++ 文件在板上首次运行时由 g++ 现场编译成动态库。最后用打包脚本合并成发布目录，经过三道校验才能出包。自己重建的位流要上板，需先改齐各处的哈希登记并逐条回读核对。

两条路径的完整命令、目录结构与逐层验证方法，见开源仓库的 README。

第六部分　总结

6.1 可扩展之处

已交付版本在三处仍有边界。多个同色目标贴在一起时，系统能发现掩膜相连，但还不能自动拆成独立目标，只能报“选择未确认”让用户重新点选；AI 服务是单工作线程加命令队列，同一时刻只处理一个请求，目标快速切换时会

排队；白书放在白底、平板边缘与桌面颜色接近时，低对比重试能救回一部分，纹理极弱时仍会丢跟踪。

更远的方向分两类。一是换相机与接口：现在走 USB 2.0，可用带宽约 35MB/s，640×480@30fps 已占去 18.4MB/s，改用 USB 3.0 或 MIPI 之后分辨率与帧率可以一并提高——校正核有 17.8 兆像素每秒的处理能力，相对相机 9.2 兆像素每秒还有近两倍余量，这条路不必改 PL。二是在现有链路上做增强，例如用亮度网格估计缓变光场做局部光补、或用网格形变处理弯曲书页；这一类要额外占 PL 资源，而 BRAM 已用到 86.4%，得先腾出余量。

6.2 设计总结

回看整个过程，收益最大的做法是“先量再改”。每次动校正核之前，先把各并行级各自的每像素占用摊开，取其中的最大值，并预先算出改完之后这个最大值是多少；如果最大值不变，这次改动就不会带来吞吐收益。这条规则是被一次失败教出来的：当时把行缓存拆成四个体，十七个测试平台全部通过，功能完全正确，吞吐却只从 7.160 变成 7.162，而 BRAM 从 96 块涨到 128 块、占整机 91%，最终布不通。回头再看，瓶颈并没有消失，只是从“等前一级”搬到了“等后一级”。

保证正确性靠的是双参考判据。每一级 RTL 都配一份独立的定点行为模型，测试平台同时比对两份参考，任何一方差异超过零即判失败；十七个测试平台覆盖单块与多块、横向与纵向、边界钳位与饱和输出。四轮迭代里，这套判据一共抓出三个只在特定几何姿态下才触发的缺陷，其中两个属于“综合能过、仿真不过”，靠上板试是试不出来的。

AI 与传统算法的分工是按代价定的。AI 只负责初始化和恢复，给出的候选先过当前帧验证，再交给跟踪模块；网络因此不进入逐帧通路，持续的推理开销被省掉。代价是首次进入 AI 路径要等一次推理，实测候选返回约 2.49 秒，明显慢于传统路径的 0.38 秒，所以它只在常规搜索失效时启用。

标注口径的统一看起来是数据工作，实际决定了整条链路能不能对上。所有样本都取目标的物理外边缘，平板把黑色边框算在内；这样模型输出的区域、跟踪器维护的边界、以及最终用于单应解算的四个角点，指的是同一个东

西。如果模型学的是内屏、跟踪跟的是边框，两者在跨帧时就会互相拉扯。

评估口径同样需要分开。定位精度、显示读数和系统稳定性是三条独立的线：扩大帧率统计窗口只改善读数的响应速度，不改变处理速度；训练集上的表现只说明拟合程度，不代表陌生场景；功能能跑起来也不等于长时间运行稳定。报告因此把这三类结论分开陈述，各自注明适用范围。

还有一条是从跨芯片流水里长出来的。PS 与 PL 在同一时刻处理的是不同的帧，一旦帧身份对不上，PL 输出的就是过期画面。系统给每一帧带上帧序号与页面代次，异步返回的结果也要回执校验，旧任务的输出直接丢弃。这看起来是工程细节，实际是「把流水做长」之后必须补上的代价——级数越多，越要能回答「这一帧是哪一帧」。

第七部分　参考文献

[1] Bradski G. The OpenCV Library[J]. Dr. Dobb's Journal of Software Tools, 2000.

[2] Howard A, Sandler M, Chu G, et al. Searching for MobileNetV3[C]//Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV). Seoul: IEEE, 2019: 1314-1324.

[3] Ronneberger O, Fischer P, Brox T. U-Net: Convolutional Networks for Biomedical Image Segmentation[C]//Medical Image Computing and Computer-Assisted Intervention (MICCAI 2015). LNCS 9351. Cham: Springer, 2015: 234-241.

[4] Tencent. ncnn: high-performance neural network inference framework[CP/OL]. https://github.com/Tencent/ncnn.

[5] Hartley R, Zisserman A. Multiple View Geometry in Computer Vision[M]. 2nd ed. Cambridge: Cambridge University Press, 2004.

[6] 交付版本软件与硬件清单[CP]. 2026-11-01.

[7] 模型训练与评估记录[Z]. 2026-11-01.

[8] run-result.log 运行记录[Z]. 2026-11-01.